Fast, efficient, VRAM-friendly LLM serving

Serving cost is decided by memory, not FLOPs

仲翊

9/4, 10:20–11:00　R546

MiRA Training Course 2026

“Put this model on our machine.” Does it fit?

Your advisor, some time in the next twelve months

“Put this model on our machine so the lab can use it.”

You have **one GPU**. The model is **70B parameters**.

- Does it fit?
- If it fits, how many people can use it at once?
- If it is too slow, what do you change *first*?

“Put this model on our machine.” Does it fit?

Your advisor, some time in the next twelve months

“Put this model on our machine so the lab can use it.”

You have **one GPU**. The model is **70B parameters**.

- Does it fit?
- If it fits, how many people can use it at once?
- If it is too slow, what do you change *first*?

Most people answer this by trial and error.

It is arithmetic, and by 11:00 you will be able to do it

The one thing

Serving cost is decided by memory, not FLOPs. Once you can compute the VRAM budget and the memory-bandwidth ceiling, every deployment decision – quantize, batch, shrink, or buy another GPU – follows from arithmetic instead of folklore.

Three questions, three pieces of arithmetic:

1. **Does it fit?** $\rightarrow$ the VRAM budget
2. **How fast can it possibly go?** $\rightarrow$ the bandwidth ceiling
3. **Which knob do I turn?** $\rightarrow$ a ranked toolbox

Every number on these slides is a cell you can run

`github.com/Chung-I/MiRA_training_course_2026`

click the Colab badge next to `notebooks/llm_serving_demo.ipynb`

https://colab.research.google.com/github/Chung-I/MiRA_training_course_2026/blob/main/notebooks/llm_serving_demo.ipynb

Parts 1–5: no GPU, no installs
Pure standard library. VRAM, KV cache, bandwidth and cost calculators. This is the arithmetic half.

Parts 6–8: needs a GPU
Colab free T4 is enough (`Runtime` → `Change runtime type`). Measures the predictions against a real card.

LIVE DEMO – notebook Part 7 and **LIVE DEMO – notebook Part 8** are run live, later.

The VRAM budget

Four things live in your VRAM, and one of them is forgotten

weights + **KV cache** + activations + framework overhead

- **Weights** – a fixed cost, paid once, easy to compute.
- **KV cache** – paid *per user, per token*. Nobody computes it.
- **Activations** – small at inference; batch-dependent.
- **Overhead / fragmentation** – allocator, CUDA context, workspace. Budget $\approx$ 2 GB and stop worrying.

Four things live in your VRAM, and one of them is forgotten

weights + **KV cache** + activations + framework overhead

- **Weights** – a fixed cost, paid once, easy to compute.
- **KV cache** – paid *per user, per token*. Nobody computes it.
- **Activations** – small at inference; batch-dependent.
- **Overhead / fragmentation** – allocator, CUDA context, workspace. Budget $\approx$ 2 GB and stop worrying.

The first two decide everything. The talk is mostly about the second.

Weights = parameters × bytes. 70B in fp16 is 140 GB.

$$\text{weight bytes} = \text{parameters} \times \text{bytes per parameter}$$

fp32 = 4 fp16 / bf16 = 2 fp8 / int8 = 1 int4 = 0.5

model	fp16	int8	int4
8B	16 GB	8 GB	4 GB
70B	**140 GB**	70 GB	**35 GB**
405B	810 GB	405 GB	203 GB

140 GB fits on no single card you own. **35 GB** fits on one 40 GB card – barely. **That one line decides the whole deployment architecture.**

The KV cache formula is the one nobody computes

$$\underbrace{\text{KV bytes per token}}_{\text{per user, per token}} = \underbrace{2}_{\text{K and V}} \times \text{layers} \times \text{kv\_heads} \times \text{head\_dim} \times \text{bytes per value}$$

- Every token you have already generated stays in memory.
- It grows *linearly* with conversation length.
- It is charged *once per concurrent user*.

Four numbers, all of them in the model's `config.json`.

Llama-3-8B: 128 KiB per token, 1 GiB per conversation

Llama-3-8B: 32 layers, 8 KV heads, head_dim 128, fp16 cache.

$$\text{KV per token} = 2 \times 32 \times 8 \times 128 \times 2 \text{ bytes}$$

Llama-3-8B: 128 KiB per token, 1 GiB per conversation

Llama-3-8B: 32 layers, 8 KV heads, head_dim 128, fp16 cache.

$$\begin{aligned}\text{KV per token} &= 2 \times 32 \times 8 \times 128 \times 2 \text{ bytes} \\ &= 131{,}072 \text{ bytes} = \textbf{128 KiB per token}\end{aligned}$$

Llama-3-8B: 128 KiB per token, 1 GiB per conversation

Llama-3-8B: 32 layers, 8 KV heads, head_dim 128, fp16 cache.

$$\begin{aligned} \text{KV per token} &= 2 \times 32 \times 8 \times 128 \times 2 \text{ bytes} \\ &= 131{,}072 \text{ bytes} = \textbf{128 KiB per token} \end{aligned}$$

$$\text{An 8k conversation} = 8192 \times 128 \text{ KiB} = \textbf{1 GiB} \text{ — for one user}$$

The weights are 16 GB once. This 1 GiB is charged again for every person who opens a chat window.

GQA is why modern models are servable at all

model	KV heads	KV per token	8k conversation
Llama-2-7B (MHA)	32	**512 KiB**	4 GiB
Llama-3-8B (GQA)	8	**128 KiB**	1 GiB

The *smaller* model costs **four times more cache per token.**

Only one thing changed: **Grouped-Query Attention** shares one set of K/V across several query heads, so `kv_heads` drops from 32 to 8. Same formula, one quarter of the memory.

Weights are a fixed cost. The KV cache is a per-user cost.

The consequence

Whatever VRAM is left *after* the weights is what you divide among concurrent users.

So the **KV cache**, not the parameter count, sets your **throughput**.

$$\text{users} \approx \frac{\text{VRAM} - \text{weights} - \text{overhead}}{\text{KV per token} \times \text{context length}}$$

One 24 GB card, one model: 7 users, or 36

Llama-3-8B, 8k context per user, 24 GB card, 2 GB overhead.

weights	KV cache	weights	free	**concurrent users**
fp16	fp16	14.9 GB	7.1 GB	**7**
int4	fp16	3.7 GB	18.3 GB	18
int4	fp8	3.7 GB	18.3 GB	**36**

The same GPU. The same model. Two configuration flags. Five times the users.

Why decoding is slow

Prefill is compute-bound. Decode is bandwidth-bound.

Prefill (your prompt)

All prompt tokens go through the network *in parallel*. Big matrix multiplies, the GPU's arithmetic units are busy. *Compute-bound.*

Decode (the answer)

One token at a time, strictly sequential. Each token requires reading *every weight* out of HBM. The GPU spends its time moving bytes. *Bandwidth-bound.*

The corollary you can charge money for

A 10k-token *prompt* is cheap. 10k *output* tokens is not. Prompt tokens are parallel; output tokens are a queue.

Your tokens/sec ceiling is bandwidth ÷ bytes of weights

$$\text{tokens/sec} \leq \frac{\text{memory bandwidth}}{\text{bytes of weights read per token}}$$

- An **upper bound**. No amount of good code beats it.
- 8B in fp16 reads **16 GB** per token; in int4 it reads **4 GB**. Same card, same math, **four times the ceiling**.

Your tokens/sec ceiling is bandwidth ÷ bytes of weights

$$\text{tokens/sec} \leq \frac{\text{memory bandwidth}}{\text{bytes of weights read per token}}$$

- An **upper bound**. No amount of good code beats it.
- 8B in fp16 reads **16 GB** per token; in int4 it reads **4 GB**. Same card, same math, **four times the ceiling**.
- Quantization speeds up decoding because it *moves fewer bytes* — not because it does less arithmetic.

Your tokens/sec ceiling is bandwidth ÷ bytes of weights

$$\text{tokens/sec} \leq \frac{\text{memory bandwidth}}{\text{bytes of weights read per token}}$$

- An **upper bound**. No amount of good code beats it.
- 8B in fp16 reads **16 GB** per token; in int4 it reads **4 GB**. Same card, same math, **four times the ceiling**.
- Quantization speeds up decoding because it *moves fewer bytes* — not because it does less arithmetic.
- Small models are *disproportionately* fast: their bottleneck is their own size.

`generate()` reaches 9.4% of the hardware

LIVE DEMO – notebook Part 6–7

RTX 5090, Qwen2.5-0.5B, fp16. First, does the arithmetic hold?

	predicted		measured
weight memory	0.920 GB	0.928 GB	(ratio **1.008**)

Then, how close to the roof does a plain for-loop get?

bandwidth ceiling	1811.6 tok/s
HuggingFace `generate()` decode	**169.9 tok/s**
efficiency	**9.4% of the hardware**

That 90% gap is the entire argument for a real serving engine.

Batching is nearly free: 15× the throughput, 4% slower per user

LIVE DEMO – notebook Part 8

The weights are read *once for the whole batch*. So a bigger batch is almost pure profit.

batch	total tok/s	per-user tok/s	wall time
1	169.3	169.3	0.39 s
16	**2594.6**	162.2	0.39 s

RTX 5090, Qwen2.5-0.5B, 64 new tokens.

- **Same wall-clock time.** Sixteen users served instead of one.
- And what limits the batch? **The KV cache** — back to Movement 1.

Report TTFT, ITL and throughput — never one average

TTFT	time to first token	prefill; what the user *feels*
ITL / TPOT	inter-token latency	decode; how fast text appears
Throughput	total tok/s, all users	what you *pay* for

- Latency and throughput *trade against each other*. Batching improves one and worsens the other.
- **A benchmark without a stated concurrency level is meaningless.**
- "Our server does 2000 tok/s" – at what batch size, and with what TTFT?

The toolbox, in order of payoff

The toolbox, ranked. The ranking is the content.

1. **Use a real serving engine.** vLLM / SGLang / TensorRT-LLM. One-line change, largest single payoff.
2. **Quantize the weights.** int4/int8/FP8. $\approx \frac{1}{4}$ the VRAM, several times the decode speed.
3. **Quantize the KV cache too.** fp8 or int8. Directly doubles your user count. Underused.
4. **Cache the prefix.** Shared system prompt or retrieved documents: prefill once, reuse.
5. Speculative decoding. 1.5–3×, identical output distribution. **CUT 2 – appendix**
6. A smaller model, or one base + many LoRA adapters. **CUT 2 – appendix**
7. More GPUs. *Last.* **CUT 2 – appendix**

Anyone can list these. The order is what you are paying for.

#1 A real engine, because two features do all the work

Continuous batching

A finished request *leaves the batch immediately* instead of waiting for the slowest one in its group. New requests join on the next step. No head-of-line blocking.

PagedAttention

The KV cache in fixed-size *pages*, like OS virtual memory. No contiguous allocation, no reserving for the worst case, no padding waste.

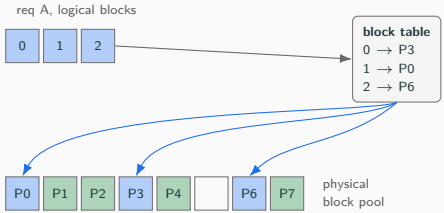
Replacing `model.generate()` in a for-loop with `vllm serve` is *a one-line change*, and it is the largest single win available to you.

PagedAttention: the KV cache as virtual memory

Contiguous: reserve for the worst case

internal fragmentation + over-reservation
prior systems used only **20.4–38.2%**
of the KV memory they allocated

Paged: fixed-size blocks, on demand



blocks need not be contiguous · waste is bounded by *one* block
identical prefixes are **shared**, copy-on-write

Kwon et al., *Efficient Memory Management for LLM Serving with PagedAttention*, arXiv:2309.06180.

#2–3 Quantize the weights, then quantize the cache

#2 Weights

AWQ, GPTQ (post-training, weight-only); FP8 natively on Hopper and newer.

$\approx \frac{1}{4}$ the VRAM at 4-bit, and several times the decode speed — because it moves fewer bytes.

#3 The KV cache

fp8 or int8 cache. Directly *doubles* the number of users you can hold.

Follows straight from Movement 1 — and almost nobody turns it on. In our table it was 18 users → **36**.

The rule

Quality loss is small on most tasks and *not* small on some.

Quantize, then re-run your own eval. Never trust a published perplexity.

#4 Prefix caching is free, and it is the biggest RAG win

- Every request begins with the same 2000-token system prompt.
- Every RAG request begins with the same retrieved documents.
- **Prefill it once. Reuse the KV blocks for every later request.**

Why it is nearly free

Those KV blocks already exist in memory, and PagedAttention already lets two requests point at the same physical block. Prefix caching is what paging *buys* you.

It costs a flag, it changes no output, and in a RAG deployment it is often the largest latency win available.

The engines themselves

vLLM is the default. Its trick is PagedAttention.

Reported in the paper (arXiv:2309.06180)

2–4× throughput · **1.7–2.7×** the request rate of Orca · up to **22×** FasterTransformer

The rest of the box, all on by default or one flag away:

- **Continuous batching** — the V1 scheduler gives each request a token budget and treats prompt and output tokens alike.
- **Hash-based prefix caching**; **chunked prefill** (on by default, decode prioritized); **CUDA graphs**.
- **Speculative decoding**: n-gram, suffix, EAGLE, MTP, draft model.
- **Quantization**: FP8 / INT8 / INT4 / GPTQ / AWQ / NVFP4, plus an **FP8 KV cache**.

SGLang's trick is RadixAttention: longest prefix, not a hash hit

Same goal as prefix caching. Different data structure.

vLLM	hash lookup: exact block match, or nothing
SGLang	**radix tree of token spans: longest-prefix match**

- A tree of *spans* finds the longest shared prefix between a new request and everything already cached — not just an exact hit.
- That structure is also what makes the cache **tierable**: you can evict a subtree instead of guessing about blocks.

If your workload is many requests sharing long, varied prefixes — agents, tool calls, few-shot prompts — this is the difference.

The tree buys tiering, constrained decoding and PD split

- **HiCache tiering.** L1 in GPU memory, L2 in host RAM, L3 in distributed storage. Your cache is no longer bounded by VRAM.
- **Constrained decoding, on a fast path.** JSON schema and regex enforced during generation. This is why SGLang is popular for *agents and tool calling*.
- **Zero-overhead CPU scheduler** with a longest-prefix-match scheduling policy.
- **EAGLE-2/3 and adaptive speculative decoding.**
- **Prefill/decode disaggregation** onto separate server groups — the two phases have opposite bottlenecks, so stop making them share a GPU. (EPD additionally splits vision encoding, for VLMs.)

TensorRT-LLM's trick is compiling ahead of time — and it bills you

The other two *interpret* your model at run time. TensorRT-LLM **builds an engine** first: kernels fused, every GEMM auto-tuned for your exact shapes and your exact GPU.

- In-flight batching, paged KV cache, custom attention backends.
- FP8 on Hopper, FP4 on Blackwell.
- TP / PP / EP / DP parallelism.

Say the cost out loud

You compile **per model, per shape range, per GPU** — and you rebuild when any of the three changes.

That is the trade: the last 20% of performance, in exchange for a build step inside your deployment pipeline.

Start with vLLM. Move only for a reason.

vLLM	the default. Start here, always.
SGLang	you live on *shared prefixes* or *structured output* (agents, tool calling, JSON schemas).
TensorRT-LLM	you ship *fixed hardware at scale* and can afford the build.

Nobody in this room should start with TensorRT-LLM.

Serving robot policies is a different problem

A robot policy is not an LLM server

	LLM server	Robot policy
wants	throughput	a *deadline*
batch size	as large as memory allows	**1, always**
latency target	hundreds of ms, on average	**20 ms, every time**
users	many, independent	one stream
failure mode	a slow reply	the robot misses its control step

Almost every trick in the last section optimizes the first column.

Continuous batching does nothing for you when the batch is always 1.

vLLM-Omni: stages, not one model — and one stage emits actions

A superset of vLLM for omni-modality: text, image, audio, video and **actions**.

- **Stage-based architecture.** A *stage* is a model component with its own execution loop, scheduling, memory and parallelism. An autoregressive stage and a diffusion/DiT stage sit in one pipeline, with overlapped execution and disaggregated resources.
- `Pi0Pipeline` — takes raw robot observations, runs flow-matching denoising, returns continuous *action chunks*.
- Served with `vllm serve --deploy-config pi0.yaml`, and it exposes an **OpenPI realtime robot endpoint** (`/v1/realtime/robot/openpi`).
- Also GR00T-N1.7, InternVLA-A1, Cosmos3 action policy.

FlashRT: no compile, hand-written kernels, one CUDA graph

The opposite design point from TensorRT-LLM — **no compilation, no ONNX export**.

- **Hand-written CUDA kernels** for the memory-bound operations: norm, activation, RoPE, quantization.
- **Delegates the compute-bound ones** to cuBLASLt / CUTLASS / FlashAttention-2.
- **The entire forward pass captured as one static CUDA graph** — zero Python dispatch at replay time.
- **Static FP8 (E4M3) and NVFP4**, calibration scales cached to disk; RMSNorm's $(1 + w)$ folded into the next GEMM at load time.

Batch size 1 means dispatch overhead *is* your latency budget.

714 ms → 44 ms was not a better model

model	hardware	latency	against
pi0.5, 2-view, FP8	RTX 5090	**17.6 ms** (57 Hz)	—
pi0.5, 2-view, FP8	Jetson AGX Thor	**44.0 ms** (23 Hz)	**13.9**× vs OpenPI reference, 714 ms
pi0, 2-view, FP8	RTX 5090	21.2 ms (47 Hz)	—
GR00T N1.6, T=50	RTX 5090	13.1 ms (76 Hz)	2.37× vs TensorRT's 31 ms

FlashRT's own reported numbers. We have not reproduced them.

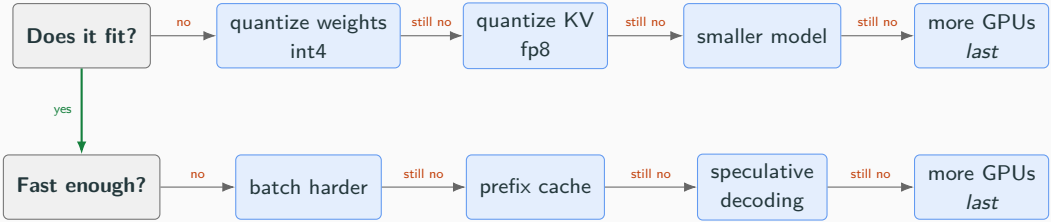
The teaching point

Quantization, kernel fusion, removing dispatch overhead — **the same three ideas as the LLM half of this talk,** aimed at a batch of one.

That is why serving is worth learning even if you only ever run robots.

Deciding, and measuring

The decision tree fits on one slide



Every arrow is cheaper than the arrow to its right.
You buy a GPU only after you have run out of arithmetic.

Dollars per million tokens — and when to just use an API

$$\$ \text{ per 1M tokens} = \frac{\text{GPU } \$/\text{hour}}{\text{tokens/sec} \times 3600} \times 10^6$$

- **tokens/sec must be your *aggregate* throughput**, across all concurrent requests — not single-stream speed. This is why batching is an *economic* decision, not an engineering one.
- An owned GPU sitting idle costs the same as a busy one.

The honest answer

Self-hosting usually *loses* to an API — unless you have **sustained high utilization**, a **fine-tuned model**, or **data that legally cannot leave the building**.

That last one is extremely common with Taiwanese manufacturing clients. It is why on-prem serving is a real job here.

How to benchmark without fooling yourself

1. **Warm up first.** The first request pays for CUDA graph capture, allocator growth and JIT.
2. **Fix the concurrency** and state it. A number without a batch size is not a number.
3. **Use realistic input *and* output lengths.** A 20-token prompt tells you nothing about a RAG deployment.
4. **Report TTFT, ITL and throughput together.** Reporting one is reporting none.
5. **Re-run the arithmetic** from Parts 1–5 against what the server reports. It should line up. If it does not, one of you is wrong and it is usually the benchmark.

Seven steps for your first deployment

1. **Compute the weight memory** before you download anything.
2. **Compute the KV cache per token** — that is your user capacity.
3. **Serve it with vLLM or SGLang.** Never a for-loop.
4. **Quantize to int4**, then re-run *your own* eval.
5. **Quantize the KV cache as well.**
6. **Cache the prefix** if your prompts share one.
7. **Report TTFT, ITL and throughput at a stated concurrency** — or report nothing.

Steps 1 and 2 need no GPU, no installs and about two minutes.

The model you can serve beats the model you cannot

The model you can serve is more useful than the model you cannot — and *which one you can serve* is decided by memory bandwidth and VRAM.

Both of which you can compute in advance.

Questions

Notebook and slides:

`https://github.com/Chung-I/MiRA_training_course_2026`

Open the serving demo directly in Colab:

`https://colab.research.google.com/github/Chung-I/MiRA_training_course_2026/blob/main/notebooks/llm_serving_demo.ipynb`

Leave this slide up. Parts 1–5 run on a free CPU runtime.

Appendix

Cut candidates, in cut order

CUT 1 – cut this first

The layer below: Triton, and a training contrast

- **Triton** — a Python DSL for GPU kernels. You write tile-level code; the compiler handles coalescing, shared memory and layout. It is what people write custom kernels in now instead of raw CUDA, and it appears *inside* every stack in this talk.
- **Liger-Kernel** — fused Triton kernels (RMSNorm, RoPE, SwiGLU, fused linear cross-entropy), claiming $+20\%$ throughput and -60% memory.

  For training, not serving. Mention it only as the contrast: the same technique, the other half of the lifecycle.

This whole frame is Movement 3d. It is the first thing to drop if the talk is running long.

CUT 2 – trim next

#5 Speculative decoding: 2×, and the same output

- A **small draft model** proposes k tokens.
- The **big model verifies all k in a single forward pass**.
- Accepted tokens are kept; the first rejection restarts from there.

Why it works — a callback to Movement 2

You are *bandwidth-bound*, so you have *spare compute*. Verifying k tokens costs roughly one token's worth of weight traffic. You are buying speed with compute you were not using.

Typically 1.5–3× faster, with an **identical output distribution** — it is not an approximation.

CUT 2 – trim next

#6 A smaller model, and #7 more GPUs, last

#6 Serve a smaller model

A distilled or simply smaller model that *passes your eval* beats a big one that is too slow to use.

And: one base model with many **LoRA adapters** instead of many full fine-tunes. The adapters are megabytes; the base model is loaded once.

(Adapters here are a serving trick. Training is another talk.)

#7 More GPUs — last

Tensor parallel inside a node; needs a fast interconnect (NVLink), or you have moved the bottleneck to the wire.

Pipeline parallel across nodes.

CPU offload as a last resort — and it is slow for exactly the same reason everything else is slow: PCIe bandwidth is more than an order of magnitude below HBM, and decode is bandwidth-bound.

Backup: the questions that always come

- **Self-host or API?** API, unless sustained high volume, a fine-tuned model, or data that cannot leave the building. Do the $/1M math.
- **Does quantization hurt quality?** Sometimes, and not uniformly across tasks. 4-bit weight-only is usually fine — confirm on *your* eval, not on perplexity.
- **Why is my model slow on a big GPU?** You are bandwidth-bound at batch 1. Batch it, or shrink the bytes.
- **vLLM, SGLang or TensorRT-LLM?** vLLM by default; SGLang for structured output and prefix reuse; TensorRT-LLM for the last 20% on fixed NVIDIA hardware.
- **How much VRAM for a 70B?** int4 $\approx$ 35 GB of weights, plus cache. In practice two 40 GB cards or one 80 GB card — and you will still want a quantized KV cache.
- **CPU offload?** You can, and it will be slow. PCIe is $> 10\times$ below HBM.
- **MoE?** Fewer *active* parameters per token, so faster decode — but you still hold *all* the experts in VRAM. Good for speed, no help for capacity.

Sources for every number on these slides

- **Llama-3-8B** 32 layers / 8 KV heads / head_dim 128; **Llama-2-7B** 32 KV heads. From the models' own `config.json` — notebook Part 2 pulls them live.
- **Capacity table** (7 / 18 / 36 users): notebook Part 3, 24 GB card, 8k context, 2 GB overhead.
- **0.920 vs 0.928 GB, 169.9 vs 1811.6 tok/s, batch 1 vs 16**: measured on an RTX 5090 with Qwen2.5-0.5B-Instruct, notebook Parts 6–8.
- **20.4–38.2% KV utilization, 2–4× throughput, 1.7–2.7× Orca, 22× FasterTransformer**: Kwon et al., arXiv:2309.06180.
- **17.6 / 44.0 / 21.2 / 13.1 ms**: FlashRT's own reported figures. Not reproduced by us.

Bandwidth figures are spec-sheet values (notebook Part 4). Prefer a number measured on the card your lab actually owns.